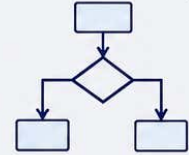
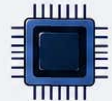


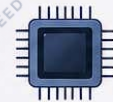
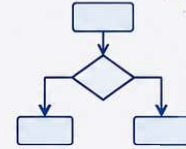
1010101010  
0101010101  
1010101010  
0101010101



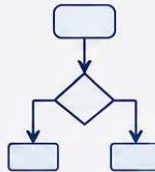
# Notes



1010101010  
0101010101  
1010101010  
0101010101



10110  
01001  
10101



# Unit-I

# Introduction to Software Engineering

## 1. Introduction:

Software Engineering is an engineering discipline that is concerned with all aspects of software production. It includes systematic approach to development, operation and maintenance of software.

## 2. Definition:

According to IEEE, "Software Engineering is the application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software."

## 3. Characteristics of Software Engineering:

- Systematic approach
- Development of Quality software
- Cost effective
- Reliability and Efficiency
- Maintainability
- Uses engineering principles

## 4. Objectives:

- To produce high quality software.
- To satisfy user requirements.
- To complete the project within time and budget.
- To ensure maintainability and reliability.

## 5. Importance of Software Engineering:

- Helps in managing large and complex projects.
- Improves quality of software.
- Reduces development time and cost.
- Provides a structured approach.
- Increases customer satisfaction.

## 6. Software Engineering Activities:

- (i) Requirement Analysis
- (ii) Design
- (iii) Coding
- (iv) Testing
- (v) Deployment
- (vi) Maintenance

These activities are performed in Software Development Life Cycle (SDLC).



# Characteristics of Software

Software has some unique characteristics which distinguish it from hardware and other engineering products. The important characteristics of software are as follows:

- ① Intangible: Software is not a physical entity. It is a collection of programs, data and documentation. We cannot touch or feel the software.
- ② Developed or Engineered, not Manufactured: Software is developed by software engineers using programming and design skills. It is not manufactured in the classical sense.
- ③ Does not Wear Out: Software does not wear out like hardware. It does not get physically damaged. However, it may become obsolete due to changes in requirements.
- ④ Custom Built: Most of the software is custom built, i.e., it is developed to meet the specific requirements of a particular user or organization.
- ⑤ Complexity: Software systems are usually complex because they have to handle large amount of data, perform various operations and interact with many users and systems.
- ⑥ Changeability: Software must be flexible and changeable. It has to evolve with changing user needs, technology and environment.
- ⑦ Invisibility: Software is invisible. We cannot see the software while it is executing. We only see its effect or output.
- ⑧ Easy Replication: Once the software is developed, it can be copied and distributed easily at very low cost.
- ⑨ Maintenance is Important: A large part of the software life cycle is spent on maintenance. It includes correcting errors, adapting to changes and improving performance.



# Types of Software

Software can be broadly classified into the following types :

1. System Software : System software is designed to manage and control the computer system hardware and to provide a platform for application software.

Examples : Operating System, Language Translators (Compiler, Interpreter), Utility Programs, Device Drivers etc.

2. Application Software : Application software is designed to help end users to perform specific tasks and solve particular problems.

Examples : MS Word, MS Excel, Accounting Packages, Payroll Software, Browser, Media Player, Mobile Apps etc.

3. Programming Software : Programming software is used by programmers to develop, test and maintain other software.

Examples : Editors, IDEs (Integrated Development Environment), Debuggers, Compilers, Libraries etc.

4. Embedded Software : Embedded software is designed to perform specific tasks within embedded systems and devices.

Examples : Software in Washing Machine, Microwave, Cars, ATM, Mobile Phones, Routers etc.

# Need of Software Engineering

Software Engineering is needed because of the following reasons :

1. Complexity of Software : Modern software systems are large, complex and involve many components. Software Engineering provides systematic methods to handle this complexity.
2. Increasing Size of Software : Software size is growing rapidly. Traditional methods are not sufficient to develop and maintain such large software.
3. Changing Requirements : User requirements keep changing. Software Engineering helps in managing changes effectively through proper planning and documentation.
4. Quality and Reliability : It ensures high quality, reliable, secure and efficient software that meets user needs and performs correctly.
5. On-time Delivery : It helps in planning, estimating and monitoring the project so that software is delivered on time.
6. Cost Effectiveness : It reduces development and maintenance cost by using proper tools, techniques and standards.
7. Maintainability : It makes the software easier to maintain, test and update in the future.
8. Better Management : It provides good project management, team coordination and communication.
9. Reusability : It promotes code reusability which saves time and effort in future projects.
10. Customer Satisfaction : It ensures that the final software meets user requirements and gives better satisfaction.

# Software Development Life Cycle (SDLC)

SDLC (Software Development Life Cycle) is a systematic process followed by software engineers to develop high quality software. It defines the various stages involved in building a software product from requirement analysis to maintenance.

## Phases of SDLC

### 1. Requirement Analysis

- In this phase, the problem is studied in detail.
- Requirements are collected, analyzed and documented.

### 2. System Design

- The requirements are converted into a design.
- It includes database design, interface design, architecture design, etc.

### 3. Implementation (Coding)

- In this phase, the actual coding is done using suitable programming languages.
- The design is converted into a working software.

### 4. Testing

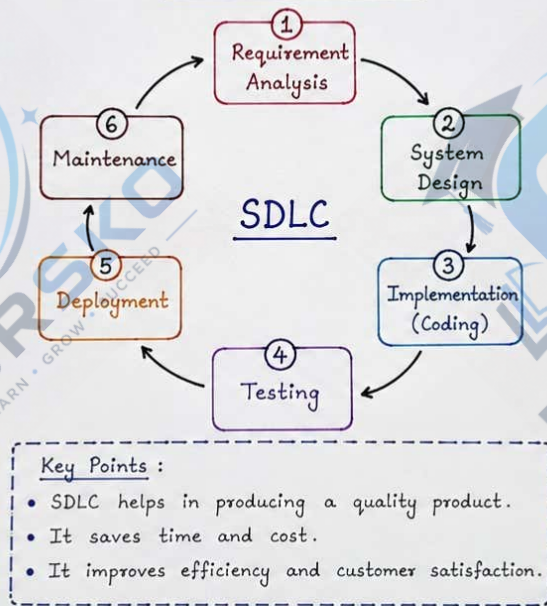
- The software is tested to find defects and errors.
- Testing ensures that the software works as per requirements.

### 5. Deployment

- After successful testing, the software is released to the user.
- It includes installation, data conversion, and user training.

### 6. Maintenance

- After deployment, the software is maintained.
- Errors are fixed, performance is improved and the software is updated according to changes in requirements.



Thus, SDLC is a continuous process and each phase is important for the successful development of a software product.



# Phases of SDLC

The Software Development Life Cycle (SDLC) consists of the following phases which are carried out in a systematic manner to develop a quality software.

- ① **Requirement Analysis** → In this phase, the problem is studied in detail. Requirements are collected from the user and documented.
- ② **System Design** → In this phase, the requirements are converted into a design. It includes database design, interface design, architecture design, etc.
- ③ **Implementation (Coding)** → In this phase, the actual coding is done using suitable programming languages. The design is converted into a working software.
- ④ **Testing** → In this phase, the software is tested to find defects and errors. Testing ensures that the software works as per the requirements.
- ⑤ **Deployment** → In this phase, after successful testing, the software is released to the user. It includes installation, data conversion and user training.
- ⑥ **Maintenance** → In this phase, after deployment, the software is maintained. Errors are fixed, performance is improved and the software is updated as per change in requirements.

# Software Development Models

A Software Development Model is an abstract representation of a software process. It provides a systematic approach to develop software.

The commonly used Software Development Models are :

## 1. Waterfall Model

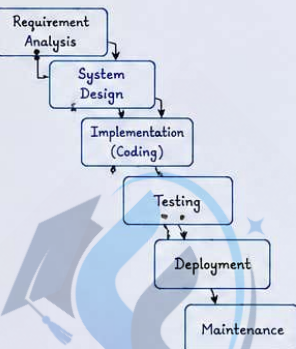
- This is a linear sequential model.
- Each phase is completed before the next phase begins.

### Advantages :

- Simple to understand
- Easy to manage

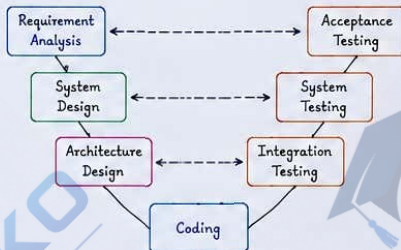
### Disadvantages :

- Inflexible
- Difficult to accommodate changes



## 2. V-Model (Verification & Validation Model)

- It is an extension of Waterfall model.
- Each development phase has a corresponding testing phase.

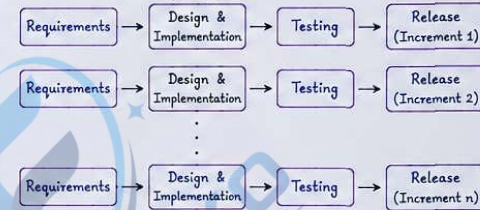


Left side → Verification (Building the product right)

Right side → Validation (Building the right product)

## 3. Incremental Model

- The software is developed in small increments.
- Each increment adds some functionality to the existing system.



### Advantages :

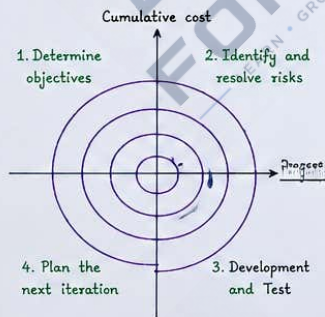
- Early delivery of working software
- Accommodates changes easily

### Disadvantages :

- Requires more planning
- System architecture must be strong

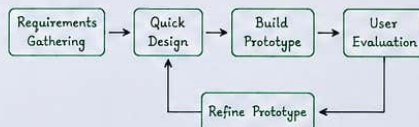
## 4. Spiral Model

- It is a risk-driven model that combines iterative development with systematic risk analysis.
- Best for large, complex and high-risk projects.



## 5. Prototype Model

- A prototype (working model) is built, tested and refined based on user feedback.
- Useful when requirements are not clear.



### Advantages :

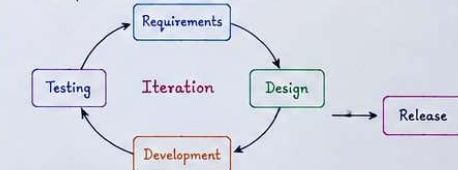
- Clarifies requirements
- User involvement

### Disadvantages :

- May lead to scope creep
- Not suitable for large systems

## 6. Agile Model

- It is an iterative and incremental model.
- Focuses on customer collaboration, flexibility and quick delivery.
- Example : Scrum, XP, Kanban



### Advantages :

- Flexible and adaptive
- High customer satisfaction

### Disadvantages :

- Needs active customer involvement
- Not suitable for fixed budget projects



# Introduction to Process Models

A software process model is an abstract representation of a process. It presents a description of a process from some particular perspective. It shows the flow of activities and their relationships. Process models are used to plan, organize and control the software development process.

## 1. What is a Process Model?

A Process Model is a simplified representation of a software process. It describes the sequence of activities involved in developing a software system and the order in which these activities are performed.

## 2. Purpose / Need of Process Models

- Provides a framework for planning and managing projects.
- Helps in estimating cost, time and resources.
- Improves quality of software.
- Facilitates communication among team members.
- Helps in monitoring and controlling the development process.
- Serves as a guide for executing the project.

## 3. Characteristics of Good Process Model

- Simple and easy to understand.
- Flexible to accommodate changes.
- Represents activities in a logical sequence.
- Defines start and end points clearly.
- Facilitates risk management.
- Improves quality and productivity.

## 4. Elements of a Process Model

A process model consists of the following elements:

- **Activities** – The work tasks performed.
- **Actions** – The specific operations within an activity.
- **Milestones** – Important points or events in the process.
- **Deliverables** – The work products produced.
- **Workflow** – The sequence and flow of activities.
- **Roles** – The people or teams involved.
- **Entry / Exit Criteria** – Conditions to start or complete an activity.

## 5. Types of Process Models

Process models can be broadly classified as:

1. **Prescriptive Process Models** – Define a specific process.
2. **Descriptive Process Models** – Describe how a process is actually performed.
3. **Empirical Process Models** – Based on observation and measurement.

## 6. Importance of Process Models

Process models play a vital role in successful software development. They help in better planning, tracking progress, managing risks and ensuring that the final product meets quality standards and customer requirements.

**Conclusion:** Process models provide a structured approach to software development. They act as a blueprint for development teams and improve quality, efficiency and predictability of the software process.

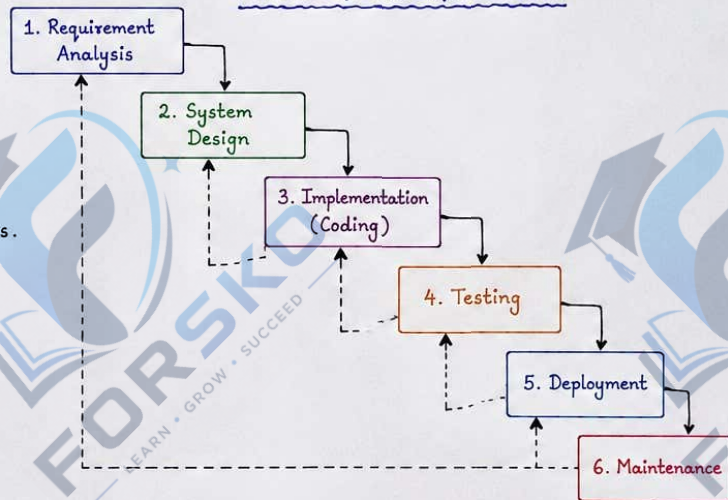
# WATERFALL MODEL

**Definition :** The Waterfall Model is a linear sequential model used in software engineering. In this model, the development process is divided into distinct phases and the output of one phase becomes the input for the next phase. Each phase must be completed before moving to the next one. It is also called a Classic Life Cycle.

## Key Points :

- It is simple and easy to understand.
- Phases are processed and completed one at a time.
- Suitable for small projects with well-defined requirements.
- Once a phase is completed, it is not easy to go back to the previous phase.

## Phases of Waterfall Model



## Explanation of Phases :

1. **Requirement Analysis :**  
Collect and analyze all requirements from the client.
2. **System Design :**  
Design the architecture, modules, database and interfaces.
3. **Implementation (Coding) :**  
Actual coding of the system according to the design.
4. **Testing :**  
Test the system to find errors and ensure it meets requirements.
5. **Deployment :**  
Deliver the software to the user and install it.
6. **Maintenance :**  
Modify and update the software after deployment to fix issues and improve performance.

## Advantages :

- Simple and easy to understand.
- Well structured and easy to manage.
- Each phase has specific deliverables.
- Works well for small projects.
- Easy to plan and control.

## Suitable for :

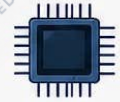
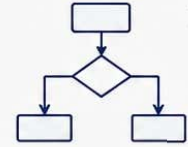
Waterfall model is suitable for projects with clear and fixed requirements, low risk, and where changes are not expected frequently.

## Disadvantages :

- Inflexible to changes.
- Working software is not produced until late in the life cycle.
- Errors found in later phases are costly to fix.
- Not suitable for large and complex projects.



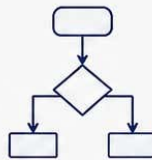
1010101010  
0101010101  
1010101010  
0101010101



# Unit - II



10110  
01001  
10101



# SOFTWARE REQUIREMENTS

Software Requirements are the capabilities, features, functions, constraints and qualities that a system must possess to satisfy the needs of stakeholders and solve a problem.

## 1. Introduction

- Requirements are the foundation of any software system.
- They describe what the system should do and the constraints under which it must operate.
- Clear and complete requirements are essential for successful software development.

## 2. Characteristics of Good Requirements (SMART)

- **Specific** – Well defined and unambiguous.
- **Measurable** – Can be measured or tested.
- **Achievable** – Realistic and attainable.
- **Relevant** – Important to stakeholders.
- **Time-bound** – Defined within a time frame.

## 3. Types of Requirements

### A. Functional Requirements

- Define the functions or services the system should provide.
- Describe what the system should do.
- Example: User login, Calculate total amount, Generate reports.

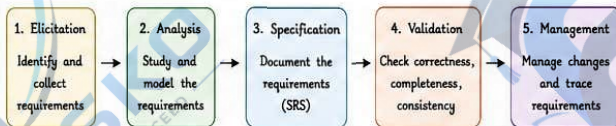
### B. Non-Functional Requirements

- Define the quality attributes and constraints of the system.
- Describe how well the system should perform.
- Examples:
  - Performance (e.g., response time < 2 sec)
  - Reliability (e.g., 99.9% uptime)
  - Usability (e.g., easy to use)
  - Security (e.g., data encryption)
  - Maintainability (e.g., easy to modify)

## 4. Sources of Requirements

- Customers / End Users
- Stakeholders
- Domain experts
- Existing systems and documents
- Laws, regulations, standards
- Market research

## 5. Requirements Engineering Process



## 6. Software Requirements Specification (SRS)

- SRS is a formal document that describes all requirements in detail.
- It acts as a contract between the customer and the development team.
- It is the basis for design, development, testing and maintenance.
- Typical SRS Contents:
  1. Introduction
  2. Overall Description
  3. Functional Requirements
  4. Non-Functional Requirements
  5. External Interface Requirements
  6. System Features
  7. Constraints
  8. Appendices

## 7. Importance of Good Requirements

- Provide a clear understanding among all stakeholders.
- Help in accurate estimation of cost and time.
- Reduce changes and rework.
- Improve quality of the software.
- Ensure customer satisfaction.

## 8. Problems due to Poor Requirements

- Misunderstandings and confusion.
- Scope creep (uncontrolled changes).
- Cost and schedule overrun.
- Low quality and customer dissatisfaction.
- Project failure.

## 9. Requirements Documentation

Requirements can be documented using:

- Natural Language
- Use Cases
- User Stories
- Data Flow Diagrams (DFD)
- Entity Relationship Diagrams (ERD)
- Prototypes
- Models

**Summary :** Software requirements define what the system should do and the constraints under which it must operate. Proper elicitation, analysis, specification and management of requirements are critical for successful software development.



# FUNCTIONAL REQUIREMENTS

Functional requirements define **what the system should do**. They describe the **functions or services** that the system must provide, the **inputs** it should accept, the **outputs** it should produce and how it should **respond** in specific situations.

## 1. Introduction

- Functional requirements specify the behavior of the system.
- They describe the services or functions the system must perform.
- They are written from the point of view of the user.
- Each requirement should express a single function or service.

## 2. Characteristics of Good Functional Requirements

- Correct** – Should reflect the real need.
- Unambiguous** – Should be clear and easy to understand.
- Complete** – All required functions should be included.
- Consistent** – No conflicting requirements.
- Verifiable** – Can be tested or demonstrated.
- Modifiable** – Easy to change if required.
- Traceable** – Each requirement should be traceable to a source.

## 3. Sources

- Users / Customers
- Stakeholders
- Domain experts
- Existing systems
- Business rules and policies
- Regulations and standards

## 4. How to Write Functional Requirements

- Use clear and simple language.
- Use active voice.
- Each requirement should start with a verb.
- Specify input, output and processing.
- Use tables, use cases or scenarios where necessary.

### Template :

The system shall [perform a function] when [certain condition] by [system response].

## 5. Examples of Functional Requirements

| No. | Requirement Description                                                              | Type                |
|-----|--------------------------------------------------------------------------------------|---------------------|
| 1   | The system shall allow a registered user to login using valid username and password. | User Authentication |
| 2   | The system shall allow the user to search for a product by name or category.         | Search              |
| 3   | The system shall allow the user to add selected products to the cart.                | Shopping Cart       |
| 4   | The system shall calculate the total amount including taxes and discounts.           | Calculation         |
| 5   | The system shall generate an order confirmation after successful payment.            | Order Processing    |
| 6   | The system shall allow the administrator to manage (add, update, delete) products.   | Admin Function      |

## 6. Types of Functional Requirements

- Input Requirements** – Data or information to be provided to the system.
- Output Requirements** – Information that the system should produce.
- Processing Requirements** – Data processing or business logic to be performed.
- Control Requirements** – Controlling the flow of activities.

## 7. Functional Requirements vs Non-Functional Requirements

| Functional Requirements                           | Non-Functional Requirements                  |
|---------------------------------------------------|----------------------------------------------|
| Define what the system does.                      | Define how well the system performs.         |
| Describe functions, services and behavior.        | Describe quality attributes and constraints. |
| Examples: Login, Search Product, Generate Report. | Examples: Performance, Security, Usability.  |

## 8. Importance

- Provide a clear understanding of system functions.
- Basis for design, development and testing.
- Help in estimation of cost and time.
- Help in validating the system by users.

**Conclusion :** Functional requirements describe the specific functions the system must perform. Well-defined functional requirements are essential for building a system that meets user needs and expectations.

# NON-FUNCTIONAL REQUIREMENTS (NFRs)

Non-functional requirements (NFRs) define **how well** the system should perform certain functions. They specify the **quality attributes**, **constraints** and **characteristics** of the system rather than specific behaviors.

## 1. Introduction

- Non-functional requirements describe the quality attributes of a system.
- They define the constraints under which the system operates.
- They affect system design, implementation, testing and maintenance.

## 2. Characteristics of Good Non-Functional Requirements

- **Measurable** - Can be measured or tested.
- **Specific** - Clearly defined and unambiguous.
- **Achievable** - Realistic and attainable.
- **Relevant** - Important to stakeholders.
- **Time-bound** - Defined for a specific period or condition.
- **Verifiable** - Can be verified or validated.
- **Consistent** - No conflict with other requirements.

## 3. Examples of Non-Functional Requirements

| Quality Attribute | Example                                                        |
|-------------------|----------------------------------------------------------------|
| Performance       | The system shall respond to user requests within 2 seconds.    |
| Reliability       | The system shall be available 99.9% of the time.               |
| Usability         | The system shall be easy to use for all categories of users.   |
| Security          | The system shall protect data from unauthorized access.        |
| Maintainability   | The system shall allow easy modifications with minimal effort. |
| Portability       | The system shall run on Windows, Linux and Mac OS.             |

## 4. Common Types of Non-Functional Requirements

| Type                                                                                              | Description                                                                                                                                                                             |
|---------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  Performance     | <ul style="list-style-type: none"><li>Response time, throughput, processing time</li><li>Example: The system shall generate reports within 5 seconds.</li></ul>                         |
|  Reliability     | <ul style="list-style-type: none"><li>Availability, fault tolerance, recoverability</li><li>Example: The system shall be available 24/7 with 99.9% uptime.</li></ul>                    |
|  Usability       | <ul style="list-style-type: none"><li>Ease of use, learnability, user interface</li><li>Example: The system shall be easy to learn and use.</li></ul>                                   |
|  Security        | <ul style="list-style-type: none"><li>Confidentiality, integrity, authentication, authorization</li><li>Example: The system shall ensure data encryption during transmission.</li></ul> |
|  Maintainability | <ul style="list-style-type: none"><li>Ease of maintenance, modularity, testability</li><li>Example: The system shall be designed in modules for easy maintenance.</li></ul>             |
|  Scalability     | <ul style="list-style-type: none"><li>Ability to handle increased load</li><li>Example: The system shall support up to 10,000 concurrent users.</li></ul>                               |
|  Portability     | <ul style="list-style-type: none"><li>Ability to run on different platforms</li><li>Example: The system shall be platform independent.</li></ul>                                        |
|  Compatibility   | <ul style="list-style-type: none"><li>Works with other systems or software</li><li>Example: The system shall be compatible with Chrome, Firefox and Edge browsers.</li></ul>            |

## 5. How to Write Non-Functional Requirements

### Template :

The system shall be [quality attribute] and shall [specific measurable criteria] under [specific conditions].

### Examples :

- The system shall response to user requests within 2 seconds under normal load.
- The system shall be available 99.9% of the time, measured monthly.
- The system shall support up to 5000 concurrent users.

## 6. Importance of Non-Functional Requirements

- Improve overall quality and user satisfaction.
- Help in making design and architectural decisions.
- Define constraints and standards for development.
- Help in testing and acceptance of the system.
- Reduce maintenance cost and increase reliability.

## 7. Difference: Functional vs Non-Functional

| Functional Requirements                  | Non-Functional Requirements                 |
|------------------------------------------|---------------------------------------------|
| Define what the system does.             | Define how well the system does it.         |
| Describe specific functions or services. | Describe quality attributes or constraints. |
| Example: User login, Add product.        | Example: Response time, Security.           |
| Visible to end users.                    | Usually invisible to end users.             |
| Change less frequently.                  | May change with environment or standards.   |

**Conclusion :** Non-functional requirements ensure that the system is not only functionally correct but also reliable, usable, secure, fast and maintainable. They play a vital role in building a high-quality software system.



# USER REQUIREMENTS

User requirements are high-level statements of **what the users (customers) need** from a system. They describe the **services, features and goals** of the system **from the user's point of view**, without focusing on **how** the system will be built.

## 1. Introduction

- User requirements describe what the users expect from the system.
- They focus on the users' tasks, goals and needs.
- They are written in natural language and are easy for all stakeholders to understand.
- They do not describe the technical or implementation details.

## 2. Characteristics of Good User Requirements

- Clear and easy to understand
- Express user needs and goals
- Independent of technology
- Feasible and realistic
- Verifiable and testable
- Unambiguous and complete

## 3. Sources of User Requirements

- End users
- Customers
- Stakeholders
- Interviews and discussions
- Questionnaires and surveys
- Observation
- Existing documents and systems

## 4. User Requirements vs System Requirements

| User Requirements                 | System Requirements                                  |
|-----------------------------------|------------------------------------------------------|
| Describe what users need.         | Describe what the system must do and how it will do. |
| Written in natural language.      | Can be in natural or formal language.                |
| High-level and general.           | Detailed and specific.                               |
| Focus on user goals and tasks.    | Focus on system functions and constraints.           |
| Do not include technical details. | Include technical and implementation details.        |

## 5. Examples of User Requirements

| No. | User Requirement Description                                             |
|-----|--------------------------------------------------------------------------|
| 1.  | The system shall allow users to register and create an account.          |
| 2.  | The system shall allow users to login securely.                          |
| 3.  | The system shall allow users to search for products by name or category. |
| 4.  | The system shall allow users to add selected products to the cart.       |
| 5.  | The system shall allow users to place an order and receive confirmation. |
| 6.  | The system shall provide users with order history and tracking.          |
| 7.  | The system shall allow users to update their profile information.        |
| 8.  | The system shall provide help and support to users.                      |

## 6. How to Write User Requirements

- Use simple and non-technical language.
- Focus on the user and their tasks.
- Use active voice and positive statements.
- Each requirement should express a single need.
- Avoid solution or design details.
- Use "shall" or "should" to state requirements.

## 7. Format / Template

The system shall [provide a service or feature] so that [user can achieve a goal or perform a task].

(This template ensures clarity and testability.)

## 8. Importance of User Requirements

- Help in understanding user needs and goals.
- Provide a basis for deriving system requirements.
- Improve communication among stakeholders.
- Reduce misunderstanding and rework.
- Ensure the final system meets user expectations and satisfaction.

**Conclusion :** User requirements are the foundation of any software development project. They capture the users' needs and expectations in simple terms and help in building a system that solves the right problem for the right users.

# SYSTEM REQUIREMENTS

System requirements are detailed descriptions of **what the system must do** and the **constraints** under which it must operate. They convert user requirements into a precise, **technical specification** that guides design, implementation, testing and maintenance.

## 1. Introduction

- System requirements describe the behavior, features, interfaces, data and constraints of the system in technical terms.
- They are the basis for system design, development, testing and validation.
- They include both functional and non-functional requirements.

## 2. Characteristics of Good System Requirements

- **Correct** - Should reflect the real need.
- **Complete** - All necessary requirements should be included.
- **Consistent** - No conflicting requirements.
- **Unambiguous** - Clear and single interpretation.
- **Verifiable** - Can be tested and validated.
- **Feasible** - Achievable with available technology and resources.
- **Necessary** - Include only essential requirements.
- **Traceable** - Traceable to user requirements.

## 3. Sources of System Requirements

- User requirements
- Stakeholders
- System analysis and modeling
- Domain experts
- Existing systems and documents
- Laws, regulations and standards
- Business rules and policies
- Market and technical constraints

## 4. Types of System Requirements

- **Functional Requirements** : Describe what the system shall do.
- **Non-Functional Requirements** : Describe how well the system shall do it and the constraints on its operations.

## 5. Components of System Requirements

| Component                                                                                                     | Description                                                                                   |
|---------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
|  Functional Requirements     | Functions, services and behaviors the system must provide.                                    |
|  Non-Functional Requirements | Quality attributes and constraints such as performance, security, reliability, usability etc. |
|  Data Requirements           | Data to be stored, processed, validated and maintained by the system.                         |
|  Interface Requirements      | Interfaces with users, other systems and hardware devices.                                    |
|  Operational Requirements    | Requirements for installation, operation, maintenance, backup and recovery.                   |
|  Design Constraints          | Constraints on standards, platforms, tools, budget, time, laws, etc.                          |

## 6. Examples of System Requirements

| No. | System Requirement Description                                           |
|-----|--------------------------------------------------------------------------|
| 1.  | The system shall allow users to register and create an account.          |
| 2.  | The system shall authenticate users using username and password.         |
| 3.  | The system shall allow users to search for products by name or category. |
| 4.  | The system shall generate invoices for confirmed orders.                 |
| 5.  | The system shall respond to user requests within 2 seconds.              |
| 6.  | The system shall ensure 99.9% availability during normal operations.     |
| 7.  | The system shall protect user data using encryption.                     |

## 7. How to Write System Requirements

- ✓ Use precise and technical language.
- ✓ Each requirement should be clear, complete and testable.
- ✓ Use active voice.
- ✓ Use "shall" or "should" for requirement statements.
- ✓ Include necessary details such as input, output, processing, data, interfaces and constraints.
- ✓ Ensure each requirement can be verified.

## 8. System Requirements Specification (SRS)

- System requirements are documented in the SRS.
- SRS acts as a contract between customer and development team.
- It is used for design, development, testing, maintenance and user training.
- **Typical SRS Contents:**
  1. Introduction
  2. Overall Description
  3. Functional Requirements
  4. Non-Functional Requirements
  5. Interface Requirements
  6. Data Requirements
  7. Design Constraints
  8. Operational Requirements
  9. Assumptions and Dependencies
  10. Appendices

## 9. Importance of System Requirements

- ◆ Provide a clear and precise understanding of the system.
- ◆ Serve as a basis for design and development.
- ◆ Help in estimating cost, time and resources.
- ◆ Help in testing, validation and quality assurance.
- ◆ Reduce risks of misunderstandings and rework.
- ◆ Ensure the final system meets user needs and business goals.

**Conclusion :** System requirements translate user needs into a detailed and testable specification. They guide the entire software development process and ensure that the system is correct, reliable, efficient and meets all constraints.



# REQUIREMENTS ENGINEERING PROCESS

Requirements engineering is the process of discovering, analyzing, documenting and managing the requirements of a software system. It ensures that the **right system** is built in the **right way**.

## 1. Introduction

- Requirements engineering is the first and most important activity in software development.
- It deals with understanding the needs of stakeholders and converting them into a complete, consistent and testable requirements specification.
- It continues throughout the life cycle of the system.

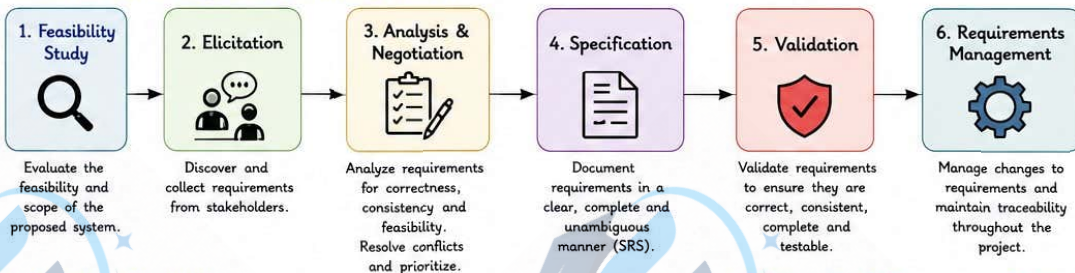
## 2. Objectives

- To understand the needs of stakeholders.
- To produce high quality requirements.
- To form a baseline for design, development, testing and validation.
- To manage changing requirements.

## 3. Key Activities

- Elicitation of requirements
- Analysis and negotiation
- Specification and documentation
- Validation
- Management of requirements

## 4. Requirements Engineering Process (Phases)



## 5. Details of Each Phase

| Phase                      | Purpose                                        | Key Activities                                                                                                                                                          | Output / Deliverables                                      |
|----------------------------|------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------|
| 1. Feasibility Study       | Determine whether the project is worth doing.  | <ul style="list-style-type: none"><li>Technical feasibility</li><li>Economic feasibility</li><li>Operational feasibility</li></ul>                                      | Feasibility Report                                         |
| 2. Elicitation             | Discover and collect requirements.             | <ul style="list-style-type: none"><li>Interviews, questionnaires</li><li>Observation</li><li>Brainstorming, workshops</li><li>Study existing documents</li></ul>        | Raw requirements (Preliminary list)                        |
| 3. Analysis & Negotiation  | Understand, model and prioritize requirements. | <ul style="list-style-type: none"><li>Classify and organize</li><li>Modeling (use cases, DFD, ERD)</li><li>Identify conflicts</li><li>Prioritize requirements</li></ul> | Analyzed and prioritized requirements                      |
| 4. Specification           | Document requirements in standard form.        | <ul style="list-style-type: none"><li>Write SRS</li><li>Use standard templates</li><li>Define functional and non-functional requirements</li></ul>                      | Software Requirements Specification (SRS)                  |
| 5. Validation              | Ensure requirements are correct and complete.  | <ul style="list-style-type: none"><li>Reviews and walkthroughs</li><li>Prototyping</li><li>Test case derivation</li></ul>                                               | Validated requirements                                     |
| 6. Requirements Management | Handle changes and maintain traceability.      | <ul style="list-style-type: none"><li>Change control</li><li>Version control</li><li>Traceability matrix</li></ul>                                                      | Requirements baseline, Change reports, Traceability matrix |

## 6. Principles of Requirements Engineering

- The customer and stakeholder are involved.
- Requirements are understandable.
- Requirements are complete, consistent and feasible.
- Requirements evolve over time.
- Requirements are traceable.
- High quality requirements are essential for success.

## 7. Benefits

- Leads to better understanding of user needs.
- Reduces rework and project risks.
- Improves communication among stakeholders.
- Provides a clear baseline for design and testing.
- Helps in delivering a system that adds real value.

## 8. Summary

The requirements engineering process ensures that we build the right system and build the system right. It is an iterative activity and continues throughout the software development life cycle.

**Note :** Good requirements = Clear + Complete + Consistent + Feasible + Verifiable + Traceable + Modifiable

# REQUIREMENTS ELICITATION AND ANALYSIS

## 1. Requirements Elicitation

Requirements elicitation is the process of discovering, gathering and understanding the **needs** and **expectations** of stakeholders for a proposed system.

### 1.1 Objectives

- To discover all possible requirements.
- To understand the real needs of stakeholders.
- To identify constraints and priorities.
- To establish a basis for further analysis and specification.

### 1.2 Techniques for Elicitation

- Interviews - One to one discussion with stakeholders.
- Questionnaires - Set of questions to collect information.
- Surveys - Collect data from large number of people.
- Observation - Observe the existing system and users.
- Workshops / JAD - Group discussion with stakeholders.
- Brainstorming - Generate as many ideas as possible.
- Document Analysis - Study existing documents, reports, manuals etc.
- Prototyping - Build a quick prototype to understand user needs.
- Use Cases - Identify actors and their goals.

**Stakeholders:** Users, Customers, Managers, Developers, Domain Experts, End Users, etc.

### 1.3 Elicitation Process



## 2. Requirements Analysis

Requirements analysis is the process of studying elicited requirements to understand, organize, model and validate them.

### 2.1 Objectives

- To classify and organize requirements.
- To check consistency, completeness and feasibility.
- To resolve conflicts and remove ambiguity.
- To prioritize requirements.
- To create a clear and structured model of requirements.

### 2.2 Activities in Requirements Analysis

- Classification - Group similar requirements.
- Organization - Structure the requirements.
- Modeling - Use models like Use Case Diagrams, Data Flow Diagrams, ER Diagrams.
- Prioritization - Identify must have, should have, could have requirements.
- Validation - Check requirements with stakeholders.
- Conflict Resolution - Resolve contradictory requirements.
- Feasibility Study - Technical, economic and operational feasibility.

### 2.3 Qualities of Good Requirements

|                |                          |
|----------------|--------------------------|
| 1. Correct     | 6. Verifiable / Testable |
| 2. Complete    | 7. Necessary             |
| 3. Consistent  | 8. Prioritized           |
| 4. Unambiguous | 9. Traceable             |
| 5. Feasible    | 10. Modifiable           |

### 2.4 Elicitation vs Analysis

| Elicitation                         | Analysis                                      |
|-------------------------------------|-----------------------------------------------|
| Discovers and gathers requirements. | Studies and understands requirements.         |
| Focus on collecting information.    | Focus on organizing and refining information. |
| Output: Raw requirements            | Output: Analyzed and structured requirements. |



# REQUIREMENTS VALIDATION

## 1. Introduction

Requirements validation is the process of checking the requirements to ensure that they are **correct, complete, consistent, feasible and testable** before system development begins.

## 2. Objectives

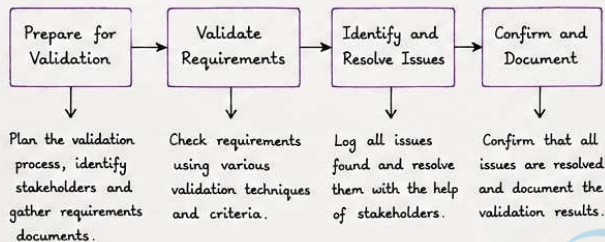
- To ensure that requirements reflect the actual needs of stakeholders.
- To detect errors, omissions, inconsistencies and ambiguities.
- To ensure that requirements are realistic, feasible and testable.
- To reduce the cost of changes by finding problems early.

## 3. Importance

- Improves quality of requirements.
- Reduces risk and rework.
- Ensures customer satisfaction.
- Provides a good baseline for design, development and testing.
- Saves time and cost.

**Note :** Finding errors in requirements later (during design or coding) is much more expensive. Therefore, validate early and validate thoroughly.

## 4. Validation Process



## 5. Validation Techniques

- Reviews - Systematic examination of requirements by reviewers.
- Walkthroughs - Author explains requirements to stakeholders.
- Inspections - Formal process with defined roles and checklists.
- Prototyping - Build a prototype to validate requirements.
- Test Case Validation - Check if requirements can be tested.
- Consistency Analysis - Check for conflicts and inconsistencies.
- Model Validation - Validate models like use case, DFD, ERD etc.
- Requirement Traceability - Trace requirements to source and downstream artifacts.

## 6. Validation Criteria (Qualities)

- |                |                          |
|----------------|--------------------------|
| 1. Correct     | 6. Verifiable / Testable |
| 2. Complete    | 7. Necessary             |
| 3. Consistent  | 8. Prioritized           |
| 4. Unambiguous | 9. Traceable             |
| 5. Feasible    | 10. Understandable       |

## 7. Activities in Validation

- ✓ Validate each requirement with stakeholders.
- ✓ Check for completeness and correctness.
- ✓ Ensure requirements are unambiguous.
- ✓ Check consistency among requirements.
- ✓ Verify feasibility with available technology, budget and time.
- ✓ Ensure requirements are testable.
- ✓ Obtain sign-off from stakeholders.

## 8. Validation vs Verification

| Validation                                 | Verification                         |
|--------------------------------------------|--------------------------------------|
| Are we building the right product?         | Are we building the product right?   |
| Focus on the correct requirements.         | Focus on the correctness of work.    |
| Done with stakeholders (customers, users). | Done by the development team.        |
| Occurs before development.                 | Occurs during and after development. |

## 9. Deliverables

- Validated Requirements Document
- Validation Report
- List of Issues and Resolutions
- Signed-off Requirements Baseline

**Conclusion :** Requirements validation ensures that the requirements are accurate, complete and acceptable to all stakeholders. It is a critical step to build the right system and avoid costly changes later.

# REQUIREMENTS MANAGEMENT

## 1. Introduction

Requirements management is the process of identifying, organizing, tracking and controlling requirements and changes to requirements throughout the project.

## 2. Objectives

- To manage changes to requirements.
- To maintain consistency and traceability of requirements.
- To ensure that requirements are complete, correct and up-to-date.
- To communicate requirements effectively among stakeholders.
- To control the impact of changes on cost, time and quality.

## 3. Importance

- Accommodates changing needs of stakeholders.
- Reduces the impact of requirement changes.
- Maintains requirement integrity.
- Improves communication and coordination.
- Ensures better planning, estimation and tracking.
- Helps in delivering the right product.

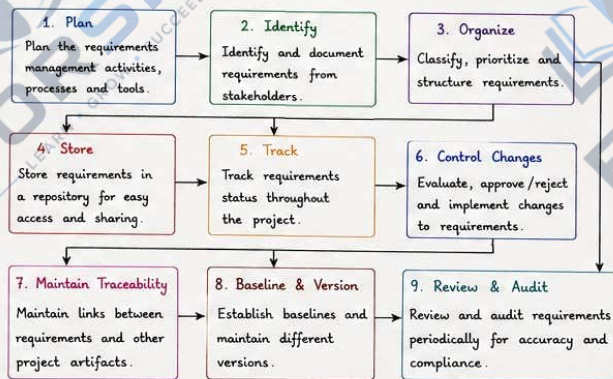
**Note :** Good requirements management ensures that the right requirements are built, changes are controlled and the final product meets stakeholder needs.

Requirements Management is the process of managing changes to requirements throughout the software development life cycle.

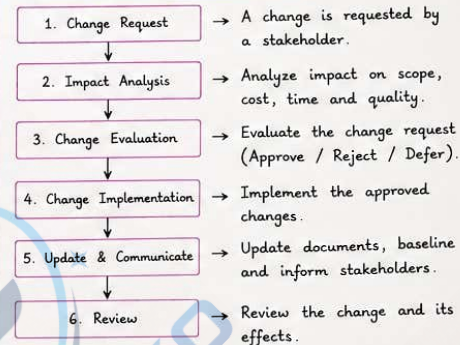
## 4. Key Activities in Requirements Management

- Requirements Identification - Identify and document requirements.
- Requirements Organization - Structure and organize requirements.
- Requirements Storage - Store requirements in a repository.
- Requirements Traceability - Maintain links among requirements and other work products.
- Change Management - Handle changes to requirements.
- Version Control - Maintain versions of requirements.
- Status Tracking - Track the status of requirements.
- Requirements Baseline - Establish and maintain baselines.
- Impact Analysis - Analyze the impact of changes.
- Requirements Audit - Review requirements for correctness and compliance.

## 5. Requirements Management Process



## 6. Change Management Process



## 7. Tools for Requirements Management

- DOORS (IBM Engineering Requirements Management DOORS)
- JIRA
- Polarion
- ReqIF
- Microsoft Excel / Word (for small projects)
- Caliber RM

## 8. Deliverables

- Requirements Management Plan
- Requirements Repository
- Requirements Traceability Matrix
- Change Request Log
- Requirements Status Report
- Requirements Baseline

**Conclusion :** Requirements management ensures that requirement changes are identified, evaluated, controlled and tracked properly. It helps in delivering a high quality product that meets stakeholder needs within time and budget.



## 1 Software Measurement

## Definition

- Software measurement is the process of assigning numbers or symbols to the attributes of a software product, process or project.

## Need / Importance

- Helps in understanding, controlling and improving the software development process.
- Provides a quantitative basis for decision making.
- Helps in estimating effort, cost and time.
- Used to monitor and control the project.
- Helps in improving product and process quality.
- Supports management in planning and evaluation.

## Purpose of Measurement

- To understand the current status.
- To estimate effort, cost and time.
- To monitor and control the project.
- To improve product and process quality.
- To support management decisions.

## Characteristics of Good Metrics

- Simple and easy to understand.
- Relevant to the objective.
- Reliable and repeatable.
- Measurable with minimum cost.
- Useful for decision making.
- Collected at the right time.
- Comparable across projects.

"What gets measured, gets managed."

- Peter Drucker

## Key Point:

Measurement is the first step towards control and improvement.

## 2 Metrics for Software Quality

## Definition

- Software quality metrics are measurable attributes used to evaluate the quality of a software product.

## Purpose

- To ensure the software meets user requirements.
- To detect and remove defects early.
- To improve maintainability, reliability and performance.
- To provide a basis for process improvement.
- To measure the overall quality of the product.

## Common Software Quality Metrics

| Metric                          | Meaning                                           | Category       |
|---------------------------------|---------------------------------------------------|----------------|
| Defect Density                  | Number of defects per KLOC (Kilo Lines of Code)   | Product Metric |
| Defect Removal Efficiency (DRE) | Percentage of defects removed before release.     | Process Metric |
| Mean Time to Failure (MTTF)     | Average time between failures.                    | Product Metric |
| Mean Time to Repair (MTTR)      | Average time to fix a failure.                    | Process Metric |
| Reliability                     | Probability of failure-free operation.            | Product Metric |
| Maintainability Index           | Measures how easy it is to maintain the software. | Product Metric |
| Cyclomatic Complexity           | Measures the complexity of code.                  | Product Metric |
| Code Coverage                   | Percentage of code tested.                        | Product Metric |

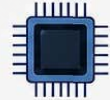
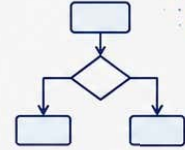
## Types of Metrics

- Product Metrics - Measure the characteristics of the software product (e.g., size, complexity, reliability).
- Process Metrics - Measure the development process (e.g., effort, time, productivity).
- Project Metrics - Measure the project performance (e.g., cost, schedule, team performance).

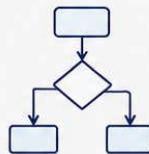
"Quality is not an act, it is a measurable result."



1010101010  
0101010101  
1010101010  
0101010101



10110  
01001  
10101





# \* History and Evolution of Java \*

## Introduction :

Java is a high level, object oriented, platform independent programming language. It was developed by Sun Microsystems. It is designed to have as few implementation dependencies as possible.

## • Origin of Java :

Java was originally developed in 1991 by James Gosling, Patrick Naughton, Mike Sheridan and their team at Sun Microsystems. The project was codenamed "Green". Initially, it was designed for small devices like digital TV, Set Top Box etc.

## • Oak to Java :

The name of the language was first "Oak", inspired by an oak tree outside James Gosling's office. Later it was renamed as "Java" in 1995. (Java means coffee in Indonesian)

Java is a name which represents strength, reliability and universal acceptance.



Java

Write Once  
Run Anywhere  
(WORA)

## Key Point :

Java evolved continuously to meet the growing needs of developers and technology.

## Evolution / Important Milestones :

- 1991 : Java project started by James Gosling at Sun Microsystems.
- 1995 : Java officially released as a programming language.
- 1996 : JDK 1.0 released.
- 1997 : JDK 1.1 released with inner classes, JavaBeans, RMI etc.
- 1998 : J2SE 1.2 released.
- 2000 : J2SE 1.3 released.
- 2002 : J2SE 1.4 released (with features like assert, regex, NIO).
- 2004 : J2SE 5.0 released (Autoboxing, Generics, Enhanced for loop).
- 2006 : Sun Microsystems was acquired by Oracle Corporation.
- 2011 : Oracle released Java 7.
- 2014 : Oracle released Java 8 (Lambda Expressions, Stream API).
- 2017 : Java 9 released (after modularization).
- 2018 : Java 10 released.
- 2021 : Java 17 (LTS version).
- 2023+ : Latest versions with improvements in performance, security and features.



# \* Features of Java \*

## Introduction :

Java is a modern programming language developed by Sun Microsystems. It has many features that make it different from other programming languages.

## Key Point :

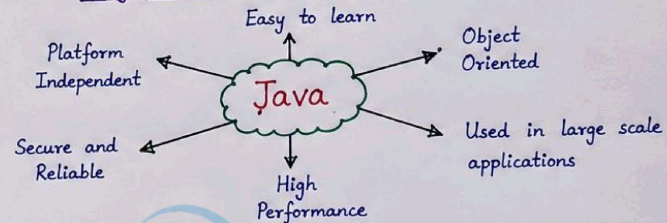
Java is designed to be simple, secure, portable and platform independent.

- ① **Simple :** → Java is easy to learn and understand. It has a simple syntax similar to C++ and removes complicated features like pointer, operator overloading, etc.
- ② **Object Oriented :** → Java is based on Object Oriented Programming (OOPs) concepts like Class, Object, Inheritance, Polymorphism, Encapsulation and Abstraction.
- ③ **Platform Independent :** → Java programs can run on any platform (Windows, Linux, Mac, etc.) without recompilation. This is achieved by JVM.
- ④ **Secure :** → Java is secure because it does not support pointer, virus free environment, and provides automatic memory management.
- ⑤ **Robust :** → Java is robust and reliable. It has strong memory management and exception handling.
- ⑥ **Portable :** → Java programs can be easily transferred from one system to another.
- ⑦ **Multithreaded :** → Java supports multithreading. It allows a program to perform multiple tasks simultaneously.
- ⑧ **Distributed :** → Java is designed for distributed environment. It provides built-in support for networking and communication.
- ⑨ **High Performance :** → Java is faster than many other interpreted languages because of bytecode and JIT (Just In Time) compiler.
- ⑩ **Dynamic :** → Java is dynamic. It supports dynamic loading of classes. It can adapt to changing environment.

## In Short :

Java is Simple, Object Oriented, Platform Independent, Secure, Robust, Portable, Multithreaded, Distributed, High Performance and Dynamic.

## Why Java is Popular ?





# \* JRE, JDK and JVM \*

## ① JRE (Java Runtime Environment)

### Definition :

JRE is an environment that provides everything required to run a Java program. It is a part of JDK.

### Components :

- JVM (Java Virtual Machine)
- Java Class Library
- Other supporting files

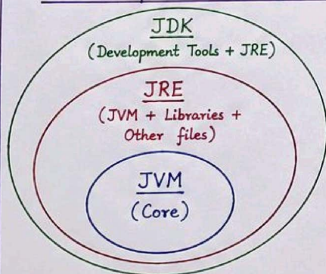
### Purpose :

It is used to run (execute) Java programs and applets.

### Key Points :

- Does not contain development tools.
- End users need JRE to run Java applications.
- It is platform dependent.

### Relationship between JDK, JRE and JVM



JDK contains JRE and development tools.

JRE contains JVM, libraries and other supporting files.

JVM is the core component.

## ② JDK (Java Development Kit)

### Definition :

JDK is a software development kit used to develop, compile, debug and run Java programs. It contains JRE and development tools.

### Components :

- JRE (Java Runtime Environment)
- Development Tools (Compiler, Debugger, Javadoc, etc.)
- Standard Class Library
- Other Utility Programs

### Purpose :

It is used by developers to create and build Java applications.

### Key Points :

- Contains JRE + Development tools.
- Required for development of Java programs.
- It is platform dependent.
- Larger in size than JRE.

### Difference Between JRE and JDK

| Feature   | JRE                           | JDK                                               |
|-----------|-------------------------------|---------------------------------------------------|
| Full Form | Java Runtime Environment      | Java Development Kit                              |
| Purpose   | To run Java programs.         | To develop, compile, debug and run Java programs. |
| Contains  | JVM + Libraries + Other files | JRE + Development Tools                           |
| Used By   | End Users                     | Developers                                        |
| Size      | Smaller                       | Larger                                            |
| Example   | To run .class file            | To create .class file                             |

## ③ JVM (Java Virtual Machine)

### Definition :

JVM is a virtual machine that acts as an intermediary between the compiled Java program (bytecode) and the operating system. It is the heart of Java.

### Working :

- ① Java code is compiled by compiler (javac) into bytecode (.class file).
- ② This bytecode is executed by JVM.
- ③ JVM converts bytecode into machine code (native code) line by line.
- ④ Then OS executes the machine code.

### Features :

- Provides platform independence.
- Memory management (automatic garbage collection).
- Security through class loader and bytecode verifier.
- Executes code in a safe environment.

### Key Points :

- JVM is part of JRE.
- It is platform dependent.
- Without JVM, Java's "Write Once, Run Anywhere" is not possible.

### In Short :

JDK is for developers, JRE is for running programs and JVM is the engine that makes Java platform independent. \*



# \* Data Types in Java \*

## Introduction :

A data type specifies the type of data that a variable can store. Java is a strongly typed language. It supports two types of data types.

Every variable in Java must be declared with a data type.

## ① Primitive Data Types (Built-in)

Primitive data types are the basic data types. They store single values.

| Data Type | Size    | Description                 | Range / Value                                           |
|-----------|---------|-----------------------------|---------------------------------------------------------|
| byte      | 1 byte  | Stores whole numbers        | -128 to 127                                             |
| short     | 2 bytes | Stores whole numbers        | -32,768 to 32,767                                       |
| int       | 4 bytes | Stores whole numbers        | -2,147,483,648 to 2,147,483,647                         |
| long      | 8 bytes | Stores whole numbers        | -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807 |
| float     | 4 bytes | Stores decimal numbers      | 1.4E-45 to 3.4028235E+38 (approx)                       |
| double    | 8 bytes | Stores decimal numbers      | 4.9E-324 to 1.7976931348623157E+308 (approx)            |
| char      | 2 bytes | Stores a single character   | \u0000 to \uFFFF (0 to 65,535)                          |
| boolean   | 1 bit   | Stores true or false values | true or false                                           |

## Examples :

```
byte b = 10;
short s = 200;
int age = 20;
long population = 780000000L;

float pi = 3.14f;
double salary = 12345.6789;
char grade = 'A';
boolean flag = true;

String name = "Saqib";
int [] marks = {85, 90, 78};
Student st = new Student();
// (user defined class)
```

## Important Points :

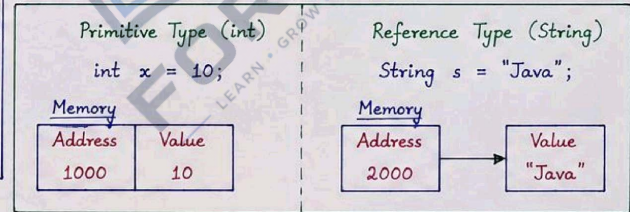
- Java is case sensitive.
- Size of data type may differ from one system to another.
- char is enclosed in single quotes.
- String is enclosed in double quotes.

## ② Non-Primitive / Reference Data Types

These are not built-in types. They store reference (address) of objects.

| Data Type | Description                          |
|-----------|--------------------------------------|
| String    | Represents a sequence of characters. |
| Array     | Stores multiple values of same type. |
| Class     | User defined data type.              |
| Interface | Blueprint of a class.                |
| Object    | Instance of a class.                 |

## Memory Representation (Example) :



## Note :

- Use 'L' or 'l' at the end of long values.
- Use 'f' or 'F' at the end of float values.
- double values do not require any suffix.
- boolean values are either true or false.



# \* Keywords in Java \*

## Introduction :

Keywords are predefined words in Java that have a special meaning. They are reserved by Java and cannot be used as identifiers (name of variables, classes, methods etc.).

## Key Points :

- They are written in lowercase.
- Keywords cannot be used as variable name, class name, method name etc.
- There are 50 keywords in Java (as per latest versions).

## Rules for Keywords :

- ① All keywords are in lowercase.
- ② They have a fixed meaning.
- ③ They cannot be used as user defined names (identifiers).
- ④ They are used to define the syntax and structure of a program.

## Example :

- ✗ `int class = 10;` → Invalid (class is a keyword)
- ✓ `int classNo = 10;` → Valid

## List of Java Keywords :

|          |            |           |              |       |
|----------|------------|-----------|--------------|-------|
| abstract | double     | int       | super        | true  |
| assert   | else       | interface | switch       | false |
| boolean  | enum       | long      | synchronized | null  |
| break    | extends    | native    | this         |       |
| byte     | final      | new       | throw        |       |
| case     | finally    | package   | throws       |       |
| catch    | float      | private   | transient    |       |
| char     | for        | protected | try          |       |
| class    | goto*      | public    | void         |       |
| const*   | if         | return    | volatile     |       |
| continue | implements | short     | while        |       |
| default  | import     | static    |              |       |
| do       | instanceof | strictfp  |              |       |

\* const and goto are reserved but not used in modern Java.

## Note :

- Java is case sensitive, but all keywords are in lowercase.
- You cannot use keywords as variable name, method name, class name etc.
- Trying to do so will give compilation error.

# Structure of a Java Program

|                                      |   |                                                                                                                      |                                                                                      |
|--------------------------------------|---|----------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| 1. Documentation Section             | → | <pre>/*<br/>This is a simple Java Program<br/>*/</pre>                                                               | } — Used to write comments about the program.                                        |
| 2. Package Statement                 | → | <pre>package myprogram;</pre>                                                                                        | → Defines the package to which the class belongs.                                    |
| 3. Import Statements                 | → | <pre>import java.util.*;<br/>import java.io.*;</pre>                                                                 | → Used to include required classes from libraries.                                   |
| 4. Interface / Class Definition      | → | <pre>public class Hello<br/>{</pre>                                                                                  | → Defines a class. Every Java program must have at least one class.                  |
| 5. Main Method                       | → | <pre>public static void main(String[] args)<br/>{</pre>                                                              | → Every Java program must have main() method. It is the starting point of execution. |
| 6. Variable Declaration              | → | <pre>int number = 10;<br/>double price = 99.99;<br/>char grade = 'A';</pre>                                          | } — Variables are declared to store data.                                            |
| 7. Executable Statements             | → | <pre>System.out.println("Hello, World!");<br/>System.out.println("Welcome to Java!");<br/>number = number + 5;</pre> | } — Statements are executed in sequence.                                             |
| 8. Return Statement / End of Program | → | <pre>return;<br/>}</pre>                                                                                             | → Indicates the end of main() method.                                                |



# Java Command Line Argument

## What is it?

Command line arguments are values passed to a Java program at the time of execution.

## How it works?

- Arguments are passed after the program name in the command prompt (or terminal).
- They are received in the main() method as an array of Strings.
- The first argument is stored at args[0], second at args[1] and so on.

## Syntax

```
public static void main(String[] args)
{
    // args is an array of strings
}
```

## Example Program

```
public class Demo
{
    public static void main(String[] args)
    {
        System.out.println("Total arguments: " + args.length);
        for (int i = 0; i < args.length; i++)
        {
            System.out.println("args[" + i + "] = " + args[i]);
        }
    }
}
```

args.length gives the number of arguments passed.

## Compilation

```
javac Demo.java
```

## Execution

```
java Demo Hello 123 "Good Morning"
```

First argument

Second argument

Third argument

Fourth argument

## Output

```
Total arguments: 4
args[0] = Hello
args[1] = 123
args[2] = Good Morning
```

## Points to remember

1. If no arguments are passed, args.length will be 0.
2. All command line arguments are treated as Strings.

Note: We can pass any number of arguments. They are separated by spaces.

# Simple Java Program

## What is it?

A simple Java program prints a message on the screen.

## Structure

### 1. Class Declaration

Every Java program must have a class.

### 2. main() Method

The execution of program starts from main() method.

### 3. Print Statement

System.out.println() is used to display output.

### 4. End of Class

Indicates the end of the class.

## Program

```
public class SimpleDemo           // Class Declaration
{
    public static void main(String[] args) // main() method
    {
        System.out.println("Hello, World!"); // Print Statement
    }                                         // End of main() method
}                                           // End of class
```

## Output

Hello, World!

→ This will be displayed on the screen.

## Steps to Run

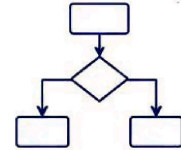
1. Write the program in a file with .java extension.
2. Compile the program using: `javac SimpleDemo.java`
3. Run the program using: `java SimpleDemo`

## Note :

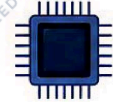
- \* Java is case-sensitive.
- \* Every statement must end with a semicolon(;;).



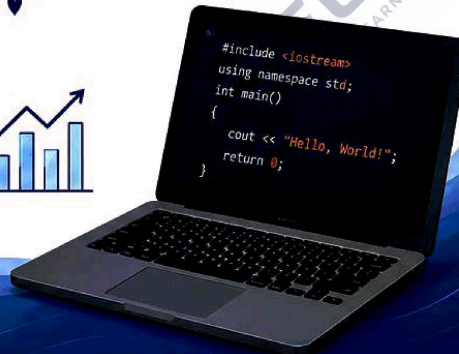
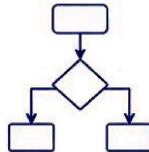
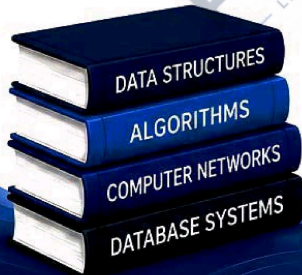
1010101010  
0101010101  
1010101010  
0101010101



# Unit - IV



10110  
01001  
10101

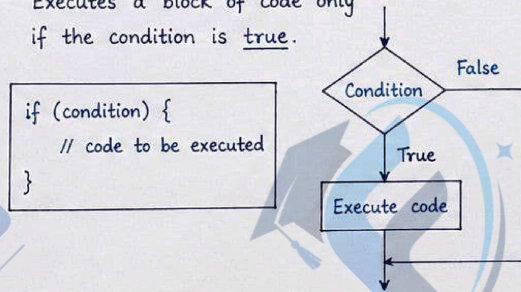


# DECISION MAKING AND BRANCHING

Decision making statements are used to execute a block of code based on a certain condition.

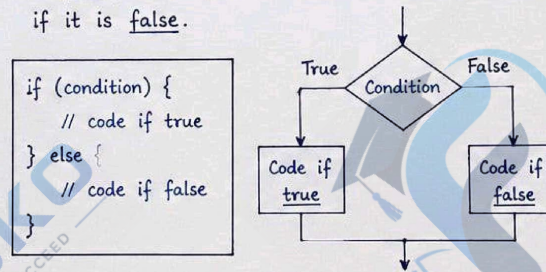
## 1) Simple if statement :

Executes a block of code only if the condition is true.



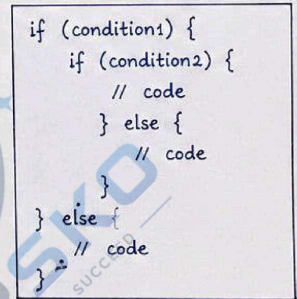
## 2) if...else statement :

Executes one block if condition is true and another block if it is false.



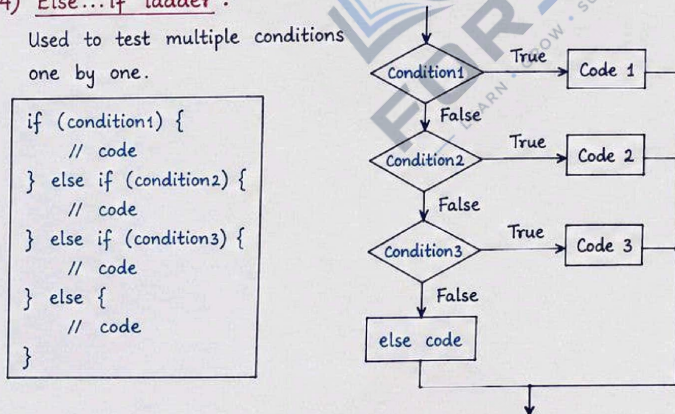
## 3) Nesting of if...else statement :

An if or else statement inside another if or else statement.



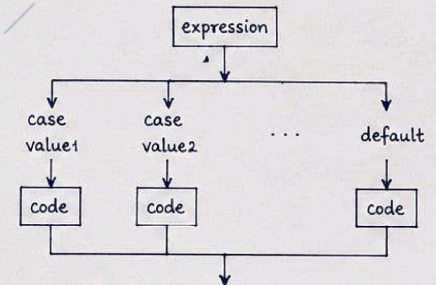
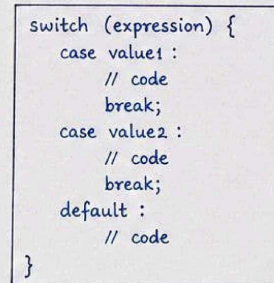
## 4) Else...if ladder :

Used to test multiple conditions one by one.



## 5) switch statement :

Used to execute one block of code from many options based on the value of a variable or expression.





# Simple if statement

## Definition :

The Simple if statement is used to execute a block of code only if the given condition is true.

## Syntax :

```
if (condition) {  
    // statements  
}
```

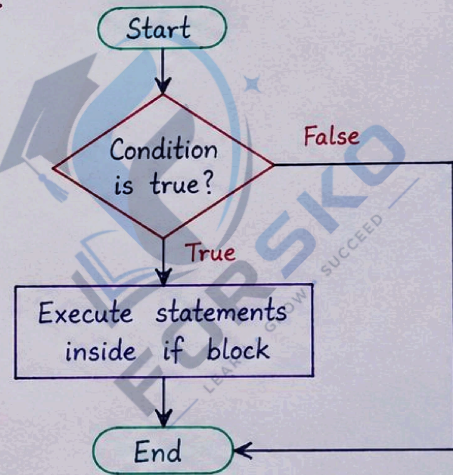
## Example :

```
public class SimpleIfExample {  
    public static void main(String[] args) {  
        int age = 20;  
        if (age >= 18) {  
            System.out.println("You are eligible to vote.");  
        }  
    }  
}
```

## How it works :

- First, the condition inside the if is evaluated.
- If the condition is true, then the block of statements inside if is executed.

## Flowchart :



## Key Point :

If the condition is false, the statements inside the if block are skipped.

# IF - ELSE STATEMENT

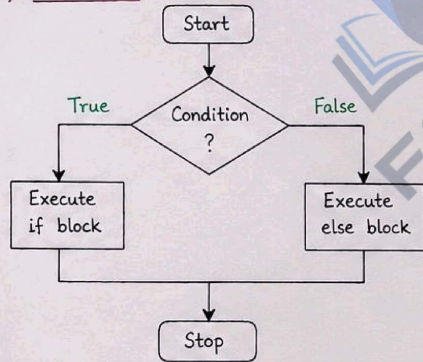
## 1) Introduction :

The if-else statement is used to execute a block of code among two alternatives.

## 2) Syntax :

```
if (condition) {  
    // statements if condition is true  
}  
else {  
    // statements if condition is false  
}
```

## 3) Flowchart :



## 4) How it works :

- First, the condition inside if is evaluated.
- if the condition is true, then the statements inside if block are executed.
- if the condition is false, then the statements inside else block are executed.

## 5) Example Program :

```
import java.util.*;  
class IfElseExample {  
    public static void main(String args[]) {  
        Scanner sc = new Scanner(System.in);  
        System.out.print("Enter your age : ");  
        int age = sc.nextInt();  
        if (age >= 18) {  
            System.out.println("You are eligible  
                                for voting.");  
        }  
        else {  
            System.out.println("You are not eligible  
                                for voting.");  
        }  
    }  
}
```

## 6) Example with working :

Input : age = 20

Condition : age >= 18 → True

Output : You are eligible for voting.

Input : age = 15

Condition : age >= 18 → False

Output : You are not eligible for voting.

## 7) Points to remember :

- if condition is a boolean expression (true/false).
- if block executes when condition is true.
- else block executes when condition is false.
- We can have nested if-else inside another if or else block.

## 8) Nested if-else Example :

```
if (marks >= 90) {  
    System.out.println("Grade : A");  
} else if (marks >= 60) {  
    System.out.println("Grade : B");  
} else {  
    System.out.println("Grade : C");  
}
```

Summary : if-else statement helps in decision making. It allows our program to choose between two paths based on a condition.

This program is created  
by SHID 😊



# NESTING OF IF - ELSE STATEMENT

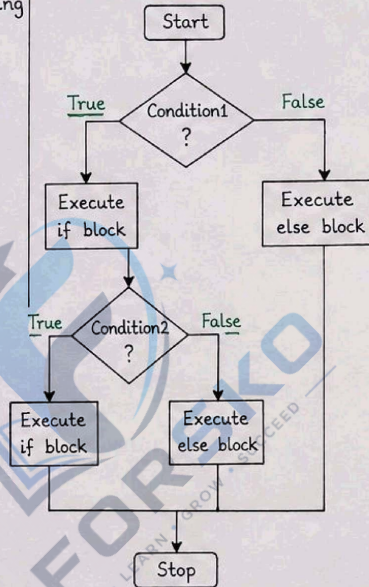
## ① Introduction :

Nesting of if-else means using one if-else statement inside another if or else block. It is used to test multiple conditions.

## ② Syntax :

```
if (condition1) {  
    // statements  
    if (condition2) {  
        // statements  
    }  
    else {  
        // statements  
    }  
}  
else {  
    // statements  
}
```

## ③ Flowchart :



## ④ Example Program :

```
import java.util.*;  
class NestedIfExample {  
    public static void main(String args[]) {  
        Scanner sc = new Scanner(System.in);  
        System.out.print("Enter marks : ");  
        int marks = sc.nextInt();  
  
        if (marks >= 90) {  
            if (marks >= 95) {  
                System.out.println("Grade : A+");  
            } else {  
                System.out.println("Grade : A");  
            }  
        } else if (marks >= 60) {  
            if (marks >= 75) {  
                System.out.println("Grade : B");  
            } else {  
                System.out.println("Grade : C");  
            }  
        } else {  
            System.out.println("Grade : Fail");  
        }  
    }  
}
```

## ⑤ Explanation :

- First, we check the outer condition (marks >= 90).
- If true, then we check inner condition (marks >= 95) inside the if block.
- If false, we go to else if (marks >= 60) and again check inner conditions.
- If all conditions are false, the else block will execute.

## ⑥ Example Output :

Enter marks : 96  
Grade : A+

-----  
Enter marks : 82  
Grade : B

-----  
Enter marks : 69  
Grade : C

-----  
Enter marks : 45  
Grade : Fail

## ⑦ Points to remember :

- We can put any number of if-else inside another if or else block.
- Proper indentation (spacing) improves readability.
- Use nested if-else only when necessary, too much nesting makes code complex.

## ⑧ Structure Example (3 levels):

```
if (condition1) {  
    if (condition2) {  
        if (condition3) {  
            // statements  
        } else {  
            // statements  
        }  
    } else {  
        // statements  
    }  
} else {  
    // statements  
}
```

← Level 1 (outer if)  
← Level 2  
← Level 3 (innermost if)

This program is created  
by SHID 😊

# ELSE IF LADDER

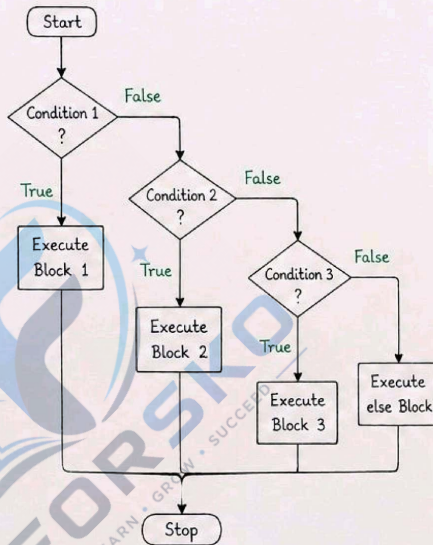
## 1) Introduction :

Else if ladder is used to check multiple conditions one by one. As soon as one condition is true, its block is executed and remaining conditions are skipped.

## 2) Syntax :

```
if (condition1) {  
    statement(s);  
}  
else if (condition2) {  
    statement(s);  
}  
else if (condition3) {  
    statement(s);  
}  
:  
else {  
    statement(s); // optional  
}
```

## 3) Flowchart :



## 4) Example Program :

```
import java.util.*;  
class ElseIfLadderExample {  
    public static void main(String args[]) {  
        Scanner sc = new Scanner(System.in);  
        System.out.print("Enter your marks :");  
        int marks = sc.nextInt();  
  
        if (marks >= 90) {  
            System.out.println("Grade : A+");  
        }  
        else if (marks >= 75) {  
            System.out.println("Grade : A");  
        }  
        else if (marks >= 60) {  
            System.out.println("Grade : B");  
        }  
        else if (marks >= 40) {  
            System.out.println("Grade : C");  
        }  
        else {  
            System.out.println("Grade : Fail");  
        }  
    }  
}
```

## 5) Explanation :

- First condition is checked.
- If it is true, its block executes.
- If it is false, next else if condition is checked.
- This continues until one condition is true.
- If none of the conditions is true, the else block executes.
- It is called a ladder because conditions are checked step by step like the steps of a ladder.

## 6) Example Output :

Enter your marks : 92  
Grade : A+

Enter your marks : 78  
Grade : A

Enter your marks : 63  
Grade : B

Enter your marks : 38  
Grade : C

Enter your marks : 25  
Grade : Fail

## 7) Points to remember :

- Conditions are checked from top to bottom.
- As soon as one condition is true, the rest are skipped.
- Use else if ladder when you have multiple conditions for one variable/expression.
- The final else block is optional but recommended.

## 8) Example (Using day number) :

```
int day = 3;  
if (day == 1) System.out.println("Monday");  
else if (day == 2) System.out.println("Tuesday");  
else if (day == 3) System.out.println("Wednesday");  
else if (day == 4) System.out.println("Thursday");  
else if (day == 5) System.out.println("Friday");  
else if (day == 6) System.out.println("Saturday");  
else System.out.println("Sunday");
```

This program is created  
by SHID 😊



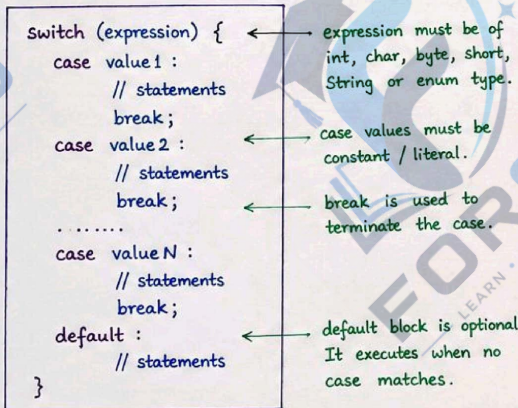
# SWITCH STATEMENT IN JAVA

## 1. Definition :

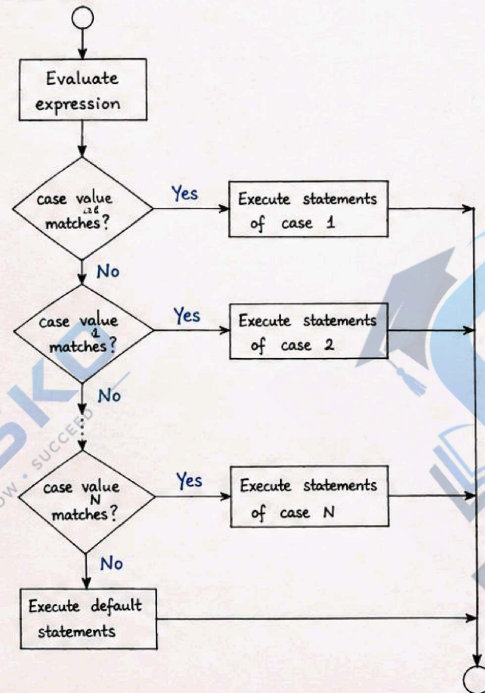
Switch statement is a multi-way decision making statement. It allows a variable to be tested for equality against a list of values.

It is an alternative to multiple if-else if statements.

## 2. Syntax :



## 3. Flowchart :



## 4. Example : Program to print day name using switch.

```
import java.util.*;  
class SwitchExample {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        System.out.print("Enter day number (1-7):");  
        int day = sc.nextInt();  
        switch (day) {  
            case 1:  
                System.out.println("Monday");  
                break;  
            case 2:  
                System.out.println("Tuesday");  
                break;  
            case 3:  
                System.out.println("Wednesday");  
                break;  
            case 4:  
                System.out.println("Thursday");  
                break;  
            case 5:  
                System.out.println("Friday");  
                break;  
            case 6:  
                System.out.println("Saturday");  
                break;  
            case 7:  
                System.out.println("Sunday");  
                break;  
            default:  
                System.out.println("Invalid day number");  
        }  
    }  
}
```

## 5. Points to Remember :

- Each case must end with break to avoid fall-through.
- If break is omitted, control will move to the next case automatically.
- default case is optional but it is good practice to handle invalid input.
- Switch improves readability when there are many possible choices.

### Output Example :

Enter day number (1-7): 3  
Wednesday

## DESIGN MAKING

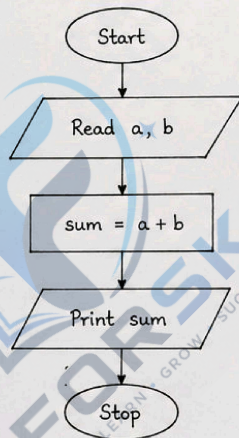
Design Making is the process of preparing the logic or steps to solve a problem before writing the program.

Example Problem: Read two numbers and print their sum.

### 1) Algorithm (Step by Step)

1. Start
2. Declare variables a, b and sum
3. Read two numbers a and b
4. Calculate  $\text{sum} = a + b$
5. Print sum
6. Stop

### 2) Flowchart



### 3) Pseudocode

```
// Program to add two numbers
BEGIN
  DECLARE a, b, sum
  READ a, b
  sum = a + b
  PRINT sum
END
```

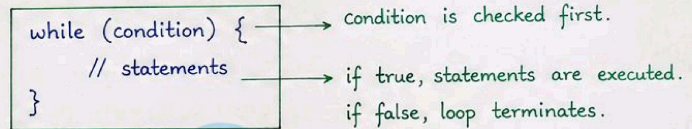
#### Note :

Good design makes the program simple, easy to understand and error free.

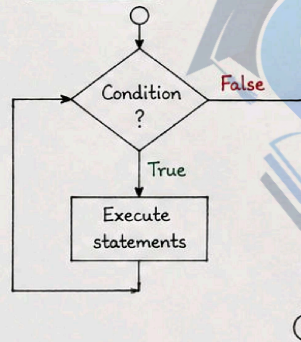
## LOOPING : while STATEMENT

while loop is an entry controlled loop. It is used to execute a block of statements repeatedly as long as the given condition is true.

### 1) Syntax :



### 2) Flowchart :



### 3) Example Program : Print numbers 1 to 5

```
class WhileExample {
  public static void main(String[] args) {
    int i = 1;           // initialization
    while (i <= 5) {      // condition
      System.out.println(i); // statement
      i = i + 1;          // update
    }
  }
}
```

#### Output :

```
1
2
3
4
5
```

#### Key Points :

- Condition is checked before executing the loop body.
- If condition is false initially, the loop body will not execute even once.

#### Note :

If we forget to update the control variable, the loop will become infinite.



# \* Do - While Statement \*

## Introduction :

do-while loop is an exit-controlled loop in Java. It executes the block of statements at least once and then checks the condition.

## Syntax :

```
do {  
    // statements  
} while (condition);
```

### Note :

Don't forget  
the semicolon  
( ; )

## Example :

```
int i = 1;  
do {  
    System.out.println(i);  
    i++;  
} while (i <= 5);
```

### Output :

1  
2  
3  
4  
5

## Special Example :

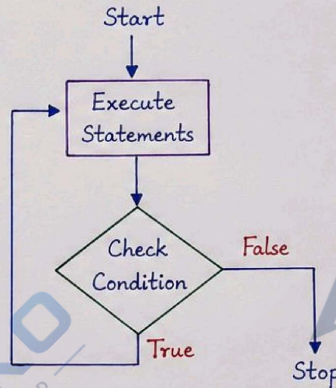
```
int i = 10;  
do {  
    System.out.println(i);  
} while (i < 5);
```

### Output :

10

Even though  
condition is false,  
the block executes  
once.

## Flow Diagram :



## Key Points :

- 1. The body of do-while loop executes at least once.
- 2. The condition is checked after the execution of the block.
- 3. If the condition is true, the statements are executed again.
- 4. If the condition is false, the loop terminates.
- 5. It is called an exit-controlled loop.

## Difference between while and do-while :

| Feature         | while Loop                             | do-while Loop                              |
|-----------------|----------------------------------------|--------------------------------------------|
| Condition check | Checked first                          | Checked after execution                    |
| Execution       | May execute 0 times                    | Executes at least 1 time                   |
| Type            | Entry-controlled loop                  | Exit-controlled loop                       |
| Syntax          | while (condition) {<br>statements<br>} | do {<br>statements<br>} while (condition); |

### Remember :

while → Check → Execute  
do-while → Execute → Check

\* Use do-while loop when the statements must be executed at least once.

## \* For Statement \*

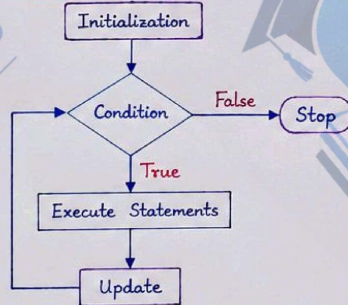
### Introduction :

The for loop is an entry-controlled loop in Java. It is used when we know in advance how many times the loop should execute.

### Syntax :

```
for ( initialization ; condition ; update ) {
    // statements
}
```

### Flow Diagram :



### Example :

```
for ( int i = 1; i <= 5; i++ ) {
    System.out.println(i);
}
```

### Output :

1  
2  
3  
4  
5

### Key Points :

- 1. Initialization is executed only once at the beginning.
- 2. Condition is checked before each iteration.
- 3. If condition is false, the loop terminates.
- 4. Update is executed after each iteration.
- 5. for loop is best when number of iterations is known.

### Note :

Any or all the three parts ( initialization, condition, update ) of for loop are optional.

## \* Jumps in Loop \*

In loops, we can change the normal flow of execution using break and continue statements.

### • break Statement :

- It is used to terminate the loop immediately.
- Control comes out of the loop.

### Example :

```
for ( int i = 1; i <= 5; i++ ) {
    if ( i == 3 )
        break;
    System.out.print(i + " ");
}
```

### Output :

1 2

Loop stops when i = 3

### • continue Statement :

- It is used to skip the current iteration.
- Control goes to the next iteration.

### Example :

```
for ( int i = 1; i <= 5; i++ ) {
    if ( i == 3 )
        continue;
    System.out.print(i + " ");
}
```

### Output :

1 2 4 5

3 is skipped, loop continues

### Difference between break and continue :

| Feature              | <u>break</u>                                                             | <u>continue</u>                                            |
|----------------------|--------------------------------------------------------------------------|------------------------------------------------------------|
| Meaning              | Terminates the loop immediately.                                         | Skips the current iteration only.                          |
| Control goes to      | Comes out of the loop.                                                   | Next iteration.                                            |
| Remaining Iterations | Not executed.                                                            | Executed.                                                  |
| Example Use          | When a specific condition is met and we don't want to continue the loop. | When we want to skip some iterations based on a condition. |

\* Remember : Use break to exit the loop completely.

Use continue to skip and move to the next iteration.